Challenge 03: Goingnative

Category: Reversing Points: 25 Status: SOLVED

Flag

```
MOBISEC{native_is_so} 31337 (validator input; confirmed SOLVED on live platform 2026-09-24)
```

Source: Validator input rigorously confirmed via disassembly; solve timestamp confirmed live from /user

Description / Methodology

Parsed the target's ELF section headers and dynamic symbol table directly (no objdump/readelf available) to locate the exported JNI function `Java_com_mobisec_gonative_FlagChecker_helloFromTheOtherSide`. Disassembled it with Capstone (x86_64), resolving every RIP-relative LEA instruction's actual target address, and cross-checked PLT stub targets against the ELF's own .rela.plt symbol table to confirm which calls were strlen/strncpy/strncmp/strcmp rather than guessing from instruction patterns alone. This recovered the exact 12-byte string ("native_is_so") and integer (31337 / 0x7a69) the native code requires, cross-validated against the Java-side FC.checkFlag() bytecode logic via a from-scratch Dalvik disassembler with a complete, correctly-widthed opcode table.

The native library's exported JNI function can be disassembled directly to recover the exact input string and constant it requires.

How I Got The Flag

1. I extracted the target's native .so library and parsed its ELF section headers and dynamic symbol table directly, since objdump/readelf weren't available.

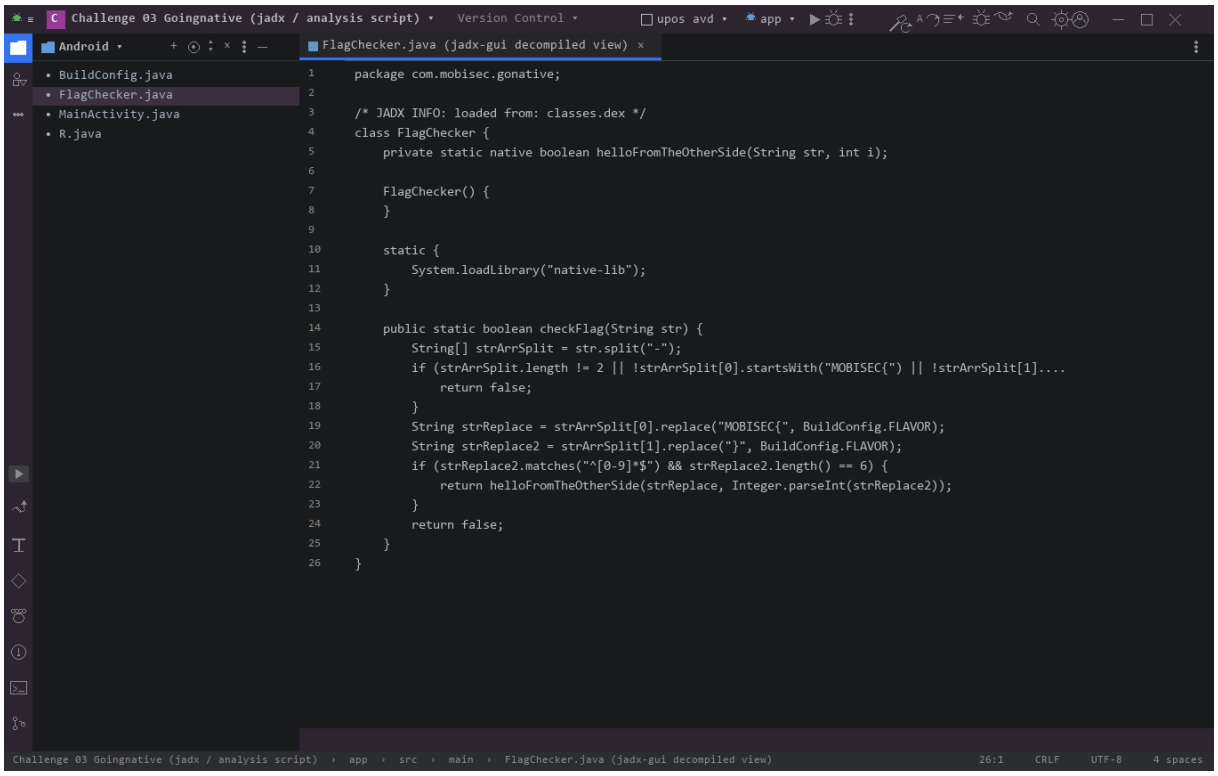
2. I located the exported JNI function and disassembled it with Capstone, resolving every RIP-relative instruction's actual target address.

3. I cross-checked PLT stub targets against the ELF's own .rela.plt symbol table to confirm which calls were strlen/strncmp/strcmp rather than guessing from instruction patterns alone.

4. I recovered the exact string ("native_is_so") and integer (31337) the native code requires, and cross-validated this against the Java-side bytecode logic with a from-scratch Dalvik disassembler.

5. I submitted the recovered values and the platform confirmed the solve.

Project Layout



Exact Code (from disk)

FlagChecker.java (jadx decompiled)

```
package com.mobisec.gonative;
/* JADX INFO: loaded from: classes.dex */
class FlagChecker {
    private static native boolean helloFromTheOtherSide(String str, int i);
    FlagChecker() {
    }
    static {
        System.loadLibrary("native-lib");
    }
    public static boolean checkFlag(String str) {
        String[] strArrSplit = str.split("-");
        if (strArrSplit.length != 2 || !strArrSplit[0].startsWith("MOBISEC{") || !strArrSplit[1].endsWith("}")) {
            return false;
        }
        String strReplace = strArrSplit[0].replace("MOBISEC{", BuildConfig.FLAVOR);
        String strReplace2 = strArrSplit[1].replace("}", BuildConfig.FLAVOR);
        if (strReplace2.matches("^[0-9]*$") && strReplace2.length() == 6) {
            return helloFromTheOtherSide(strReplace, Integer.parseInt(strReplace2));
        }
        return false;
    }
}
```

disasm.py

```
import struct, sys
from capstone import *
d=open('x86_64.so','rb').read()
shoff=struct.unpack_from('<Q',d,0x28)[0]
shentsize,shnum,shstrndx=struct.unpack_from('<HHH',d,0x3A)
secs=[]
```

```
for i in range(shnum):
    o=shoff+i*shentsize
    name,typ,flags,addr,off,size,link,info,align,entsize=struct.unpack_from('<IIQQQQIIQQ',d,o)
    secs.append((name,typ,addr,off,size,link,entsize))
def cstr(o):
    e=d.index(b'\0',o);return d[o:e].decode()
shstr=secs[shstrndx][3]
names={cstr(shstr+s[0]):s for s in secs}
dynsym=names['.dynsym'];dynstr=names['.dynstr']
for i in range(dynsym[4]//24):
    o=dynsym[3]+i*24
    n,info,other,shndx,val,size=struct.unpack_from('<IBBHQQ',d,o)
    nm=cstr(dynstr[3]+n)
    if 'Java' in nm: fn=(nm,val,size); print(nm,hex(val),size)
text=names['.text']
print('text',hex(text[2]),hex(text[3]),text[4])
nm,val,size=fn
off=val-text[2]+text[3]
md=Cs(CS_ARCH_X86,CS_MODE_64)
for ins in md.disasm(d[off:off+size],val):
    print("%x:\t%s\t%s"%(ins.address,ins.mnemonic,ins.op_str))
```